Processo Seletivo - ESIG Group

Estagio QA - Automação

Fundamentos de Testes

Documento Teórico

Projeto: Quark Clinic - Portal de Agendamento Online

Candidato: Matheus Reinaldo

Data: 17 de abril de 2026

Introdução

Este documento foi produzido como parte da atividade do processo seletivo para Estagio em QA - Automação na ESIG Group. Ele reúne os fundamentos teóricos de testes aplicados diretamente ao projeto de automação E2E, desenvolvido para o portal de agendamento da Quark Clinic.

Os conceitos aqui apresentados não são apenas definições acadêmicas. Cada um deles foi vivenciado na prática durante a implementação dos fluxos automatizados, com isso, foi possível conectar a teoria com decisões e situações reais que enfrentei ao longo do desenvolvimento do projeto.

.

1. Plano de Testes

1.1 O que é, e qual sua importância?

Antes de escrever a primeira linha de código de teste neste projeto, precisei entender o que seria testado, como seria testado e quais eram os critérios de sucesso. Essa reflexão é exatamente o que um Plano de Testes formaliza.

Um Plano de Testes é um documento que define o escopo, a abordagem, os recursos e os critérios de uma atividade de teste. Ele funciona como um contrato entre a equipe de QA, os desenvolvedores e os stakeholders, garantindo que todos tenham a mesma visão sobre o que precisa ser validado.

Sua importância fica evidente quando pensamos nos riscos de não o ter: sem um plano claro, é fácil desperdiçar tempo testando funcionalidades de baixo risco enquanto fluxos críticos ficam sem cobertura. No contexto da Quark Clinic, por exemplo, um agendamento que falha silenciosamente representa um paciente sem consulta, um impacto direto no negócio.

1.2 Seções-chave para o projeto Quark Clinic

Seção	Descrição	Aplicação no Quark Clinic
Objetivo	Proposito geral dos testes	Garantir que o fluxo completo de agendamento funcione corretamente para o paciente — do cadastro ao envio do comprovante
Escopo	O que entra e o que fica fora	Em escopo: cadastro, login, agendamento presencial, comprovante. Fora do escopo: painel administrativo, teleconsulta, exames
Abordagem	Estratégia e técnicas utilizadas	Testes E2E automatizados com Cypress, usando cy.intercept() para validar requisições de rede sem esperas estáticas
Ambiente	Infraestrutura de testes	Chrome + Cypress 15.4.0, ambiente de homologação da Quark Clinic
Dados de Teste	Massa de dados utilizada	Dados dinâmicos gerados com @faker-js/faker para cadastro; fixture usuario.json para login e agendamento
Critérios de Entrada	Quando iniciar os testes	Ambiente de homologação estável, fluxos de cadastro e login implementados e navegáveis
Critérios de Saida	Quando encerrar os testes	100% dos fluxos críticos passando, sem erros bloqueantes, intercepts validando status 200
Riscos Identificados	Possíveis impedimentos	Horários dinâmicos na agenda com timestamps que mudam diariamente, extensão incorreta de arquivos de upload, comportamentos não documentados na interface

2. Tipos de Testes

Durante o desenvolvimento deste projeto, trabalhei predominantemente com a abordagem de Caixa Cinza, mas entender os três tipos foi fundamental para tomar as decisões certas em cada etapa.

2.1 Teste de Caixa Preta (Black Box)

No teste de Caixa Preta, o testador avalia o comportamento do sistema apenas pelas suas entradas e saídas, sem qualquer conhecimento do código interno. E a perspectiva do usuário final, o que importa e se o sistema faz o que promete, independentemente de como faz.

Características principais:

- Nenhum conhecimento da implementação interna e necessário
- Baseado inteiramente em requisitos e especificações funcionais
- Técnicas comuns: partição de equivalência, analise de valor limite, tabela de decisão

> *Exemplo real no projeto: Durante a exploração inicial do portal, testei manualmente os formulários sem saber nada sobre a API por trás. Descobri que o campo de celular aceitava qualquer formato na interface, mas ao inspecionar a requisição de rede, percebi que havia validação no backend. Isso e Caixa Preta pura: avaliar o comportamento visível sem ver o código.*

2.2 Teste de Caixa Branca (White Box)

No teste de Caixa Branca, o testador tem acesso total ao código-fonte e testa os caminhos internos da aplicação. O objetivo e garantir que cada condicional, loop e função seja exercitado pelos testes.

Características principais:

- Exige conhecimento de programação e da arquitetura interna
- Foca em cobertura de código, instruções, branches e caminhos
- Tipicamente executado pelos próprios desenvolvedores ou QA técnico

> *Exemplo aplicado ao projeto: Se tivéssemos acesso ao backend da Quark Clinic, um teste de Caixa Branca verificaria diretamente a função de validação de CPF, testando os algoritmos dos dígitos verificadores e casos extremos que não são visíveis pela interface, como CPFs com todos os dígitos iguais.*

2.3 Teste de Caixa Cinza (Gray Box)

O teste de Caixa Cinza combina as duas abordagens anteriores. O testador conhece parcialmente a estrutura interna, especialmente os contratos de API e a arquitetura, sem necessariamente ter acesso ao código-fonte completo. E a abordagem ideal para testes E2E bem feitos.

Características principais:

- Conhecimento parcial: endpoints de API, estrutura do banco, fluxo de dados
- Permite criar testes mais precisos e direcionados a pontos críticos
- Ideal para testes de integração, E2E e validação de APIs

> *Exemplo central do projeto: Ao inspecionar as requisições de rede com o DevTools, identifiquei os endpoints reais da API - POST /api/social/usuarios para cadastro, POST /api/auth/login para login e POST /api/protected/me/agendamentos para agendamento. Com esse conhecimento, usei cy.intercept() para interceptar cada requisição e validar o retorno, sem precisar do código-fonte. Isso e Caixa Cinza: usar o conhecimento da API para criar testes E2E mais robustos.*

Aspecto	Caixa Preta	Caixa Cinza	Caixa Branca
Conhecimento interno	Nenhum	Parcial (API, arquitetura)	Total (código-fonte)
Perspectiva	Usuário final	QA técnico com acesso a API	Desenvolvedor
Ferramentas típicas	Exploratório, casos manuais	Cypress com cy.intercept()	Jest, JUnit, code coverage
Uso neste projeto	Exploração inicial do portal	Automação E2E com interceptação	Não aplicado (sem acesso ao backend)

3. Casos de Teste

3.1 O que são Casos de Teste?

Um Caso de Teste e um conjunto estruturado de condições, passos e critérios de validação que permite verificar se uma funcionalidade especifica se comporta conforme o esperado. Cada caso documenta: qual é a situação inicial, quais ações são executadas e qual deve ser o resultado.

Neste projeto, os próprios testes automatizados funcionam como casos de teste executáveis, cada it() e um caso com sua pré-condição (beforeEach), ações (cy.get, cy.type, cy.click) e validações (cy.contains, cy.should). Os três casos manuais abaixo complementam essa cobertura com cenários que vão além do caminho feliz.

Um bom caso de teste deve ser:

- Claro: qualquer membro da equipe consegue executa-lo sem ambiguidade
- Rastreável: vinculado a um requisito ou funcionalidade especifica
- Repetível: produz o mesmo resultado em execuções diferentes
- Independente: não depende do resultado de outros casos de teste

3.2 Por que o fluxo de Login?

O fluxo de Login foi escolhido porque, durante a automação, encontrei um comportamento não documentado que revelou a riqueza de cenários possíveis nessa tela: o sistema possui um check-box de Política de Privacidade obrigatório para que a requisição de login seja enviada. Essa descoberta só aconteceu quando o teste automatizado falhou, logo o cy.wait travou esperando uma requisição que nunca saiu, porque o checkbox não estava marcado.

Esse momento gerou o CT-LOGIN-03, um cenário de borda que provavelmente não seria escrito sem a experiencia pratica com o sistema real.

3.3 Casos de Teste em formato BDD

Os casos abaixo utilizam a sintaxe Gherkin (Given/When/Then), padrao do BDD (Behavior-Driven Development). Essa abordagem descreve o comportamento em linguagem natural estruturada, facilitando a comunicação entre QA, desenvolvimento e negócio.

CT-LOGIN-01: Login com credenciais validas (Cenário Positivo)

Campo	Descrição
ID	CT-LOGIN-01
Funcionalidade	Login de usuário
Tipo	Positivo, caminho feliz

Campo	Descrição
Pré-condição	Existe um usuário previamente cadastrado no sistema com e-mail e senha validos
Prioridade	Alta, e o fluxo de entrada para todos os outros fluxos críticos
Cobertura automatizada	Sim, arquivo 02-login.cy.js utilizando cy.loginComUsuario()

Cenário: Login bem-sucedido com credenciais validas

Dado que o usuário acessa o portal de agendamento da Quark Clinic
E clica no botão "Login" no canto superior direito
E o modal de login e exibido com os campos Usuário, Senha e o checkbox de politica

Quando o usuário preenche o campo "Usuário" com um e-mail valido cadastrado no sistema
E preenche o campo "Senha" com a senha correspondente
E marca o checkbox "Eu concordo com a Política de Privacidade e com o Termo de Uso"
E clica no botão "Continuar"

Então o login e processado com sucesso
E o usuário e redirecionado para a home page logada
E a saudação "Bem-vindo(a), [Nome do Usuário]" e exibida no cabeçalho

CT-LOGIN-02: Login com senha incorreta (Cenário Negativo)

Campo	Descrição
ID	CT-LOGIN-02
Funcionalidade	Login de usuário, validação de credenciais
Tipo	Negativo, credencial invalida
Pré-condição	Existe um usuário previamente cadastrado no sistema com e-mail valido
Prioridade	Alta, segurança básica do sistema
Cobertura automatizada	Não, candidato a próxima iteração do projeto

Cenário: Tentativa de login com senha incorreta

Dado que o usuário acessa o portal de agendamento da Quark Clinic
E clica no botão "Login" no canto superior direito
E o modal de login e exibido

Quando o usuário preenche o campo "Usuário" com um e-mail valido cadastrado no sistema
E preenche o campo "Senha" com uma senha incorreta
E marca o checkbox de política de privacidade
E clica no botão "Continuar"

Então uma mensagem de erro e exibida ao usuário na interface
E o usuário permanece na tela de login sem ser redirecionado
E os campos permanecem preenchidos para que o usuário possa tentar novamente

CT-LOGIN-03: Login sem marcar o checkbox de política (Cenário de Borda)

Campo	Descrição
ID	CT-LOGIN-03
Funcionalidade	Login de usuário, validação de campo obrigatório
Tipo	Borda, comportamento não obvio descoberto durante a automação
Pré-condição	Nenhuma, pode ser executado sem usuário previamente cadastrado
Prioridade	Media, impacto direto na experiencia do usuário
Cobertura automatizada	Parcial, corrigido e incorporado ao cy.loginComUsuario() no Fluxo 2

> *Origem deste cenário: Durante a automação do Fluxo 2, o cy.wait travou esperando uma requisição que nunca foi enviada. Ao inspecionar a tela, percebi que o sistema bloqueia o envio da requisição enquanto o checkbox não estiver marcado. Esse comportamento não estava documentado, foi descoberto pela falha do teste automatizado.*

Cenário: Tentativa de login sem aceitar os termos de uso

Dado que o usuário acessa o portal de agendamento da Quark Clinic
E clica no botão "Login" no canto superior direito
E o modal de login e exibido

Quando o usuário preenche o campo "Usuário" com um e-mail valido
E preenche o campo "Senha" com a senha correta
E NAO marca o checkbox "Eu concordo com a Política de Privacidade e com o Termo de Uso"
E clica no botão "Continuar"

Então nenhuma requisição de login e enviada ao servidor

E a mensagem "E necessário informar que concorda com a Política de Privacidade" e exibida

E o usuário permanece na tela de login

3.4 Resumo dos Casos de Teste

ID	Titulo	Tipo	Prioridade	Automatizado
CT-LOGIN-01	Login com credenciais validas	Positivo	Alta	Sim, 02-login.cy.js
CT-LOGIN-02	Login com senha incorreta	Negativo	Alta	Não, próxima iteração
CT-LOGIN-03	Login sem aceitar os termos	Borda	Media	Parcial, corrigido no Fluxo 2

4. Lições Aprendidas e Decisões Técnicas

Esta seção documenta as principais situações reais que enfrentei durante o desenvolvimento, e o que aprendi com cada uma delas. Esse registro e valioso porque documenta o raciocínio por trás das decisões, não apenas o resultado final.

4.1 cy.intercept() vs cy.wait()

Uma das primeiras decisões técnicas foi escolher como lidar com a assincronia da aplicação. A opção mais simples seria cy.wait(3000), mas essa abordagem tem problemas sérios:

- Torna os testes lentos e arbitrários, visto que 3 segundos podem ser considerados pouco ou muito dependendo da conexão
- Os testes ficam frágeis (flaky) passam as vezes e falham outras sem motivo aparente
- E explicitamente listado como má pratica nos critérios de avaliação do teste

> *A solução foi cy.intercept() combinado com cy.wait('@alias'). O intercept registra um espião de rede antes da ação, o Cypress só avança quando a requisição real acontece e retorna. Para descobrir a URL correta de cada endpoint, inspecionei as requisições reais na aba Network do DevTools. Sem esse passo, o intercept não captura nada.*

4.2 Seletores data-cy — o padrão certo para automação

Ao inspecionar o portal com o DevTools, percebi que o time de desenvolvimento já havia adicionado atributos data-cy em praticamente todos os elementos interativos. Esses seletores foram criados especificamente para testes e tem vantagens claras:

- Não mudam com refatorações de estilo, um redesign visual não quebra os testes
- São semanticamente claros, data-cy='btn-confirmar' e mais legível que .btn-primary.my-3.w-100
- São a recomendação oficial da documentação do Cypress para seletores resilientes

4.3 Bugs encontrados e corrigidos durante o desenvolvimento

Bug	Sintoma	Causa Raiz	Correção Aplicada
Select com valor incorreto	cy.select() failed , no option matching: M	O select de Sexo usa 'MASCULINO' em maiúsculo, não 'M'	cy.get('#sexo').select('MASCULINO')

Bug	Sintoma	Causa Raiz	Correção Aplicada
URL do intercept errada	cy.wait() timed out , No request ever occurred	O padrão não correspondia a URL real da API	Inspeção da aba Network para descobrir a URL real de cada endpoint
Checkbox obrigatório no login	Requisição de login nunca enviada	Sistema exige aceite dos termos, comportamento não documentado	Adição do. check({ force: true }) no checkbox, originou o CT-LOGIN-03
Case-sensitive no assert	Expected: 'sucesso!' - Received: 'Sucesso!'	Texto real usava S maiúsculo em 'Sucesso!'	Correção do texto exato no cy.contains()
Arquivo com extensão dupla	selectFile failed - file does not exist	Arquivo salvo como comprovante.png.jpg pelo Windows	Renomeação correta para comprovante.png
Timestamps dinâmicos na agenda	Seletor fixo não encontrava o horário	data-cy dos horários usa timestamps Unix que mudam diariamente	Uso do operador ^= com .first() para pegar o primeiro horário disponível
Código duplicado após refatoração	Declaração ou instrução esperada - erro de sintaxe	Novo código adicionado sem remover o código antigo	Limpeza do arquivo mantendo apenas cy.loginComUsuario()

4.4 Comando reutilizável cy.loginComUsuario()

Os fluxos 2, 3 e 4 precisavam de login como pré-condição. Repetir o código de login em cada arquivo violaria o princípio DRY (Don't Repeat Yourself) se a tela de login mudasse, seriam três lugares para atualizar.

> *A solução foi criar um comando customizado no arquivo cypress/support/commands.js: Cypress.Commands.add('loginComUsuario', () => { ... }). Com isso, qualquer teste usa apenas cy.loginComUsuario() uma linha. O arquivo 02-login.cy.js ficou com 4 linhas no total. Isso e o que separa um código de teste amador de um profissional: legibilidade e facilidade de manutenção.*

5. Conclusão

Este projeto me permitiu aplicar na pratica conceitos que até então conhecia apenas teoricamente. A experiencia de ver um teste falhar, identificar a causa raiz e corrigi-la foi mais enriquecedora do que qualquer leitura sobre o assunto.

Os 4 fluxos automatizados: cadastro, login, agendamento e envio de comprovante, cobrem o caminho critico de um paciente no portal da Quark Clinic. Cada um utiliza boas práticas de

automação: seletores data-cy, interceptacao com cy.intercept(), dados dinâmicos com Faker.js e comandos reutilizáveis.

Mais do que testes que passam, o objetivo foi construir uma suíte de testes confiável, legível e de fácil manutenibilidade, características que fazem a diferença em um ambiente de trabalho real.